

HW3: Classification

Group : G4

Contributors

- | | |
|--------------------------|--------------|
| 1. Mr. Thananop Kullapan | 653-01821-21 |
| 2. Mr. Kanawat Suwandee | 687-00268-21 |
| 3. Mrs. Yuwadee Tongkong | 687-20736-21 |
-

Your tasks are:

Download the Google sheet and answer all the questions. Please download as a PDF file and rename it from 'Lab3.pdf' to 'Lab3_G1.pdf'.

Go to Github [Lab3](#) and [Google Doc](#)

Goal:

1. can implement and train a **neural network**.
2. can use the evaluation metrics and validation techniques for choosing the best configuration of neural networks.
3. understand how to use early stoppers.

Assignment sheet

1. Explain how an early stopper works. You can do this by providing the flowchart and use the example in the [python notebook](#).
2. Explain how you will extend the neural network and select the different neural network configurations using validation technique.
 - a. Provide the code to extend the number of layers to 1, 2, 3.
 - b. Plot F1 / BCE loss curves versus the number of layers. Then, explain which you would use.

The code is implemented in [Lab3/main.ipnb](#). This notebook already provided:

1. Tools and packages
2. Downloaded dataset from [kaggle](#)
3. Data processing

Details are provided in [Lab3/readme.md](#).

Please answer these questions using your results to provide the discussion. Make sure that every point that you made is supported by the evidence and the data.

Question 1

Explain how an early stopper works. You can do this by providing the flowchart and use the example in the [python notebook](#).

Answer

Early Stopping is a machine learning technique used to prevent **overfitting**—when a model memorizes training data instead of learning general patterns. It works by monitoring the model's performance on unseen validation data and automatically halting training as soon as the performance stops improving. This ensures the model remains accurate and generalizable.

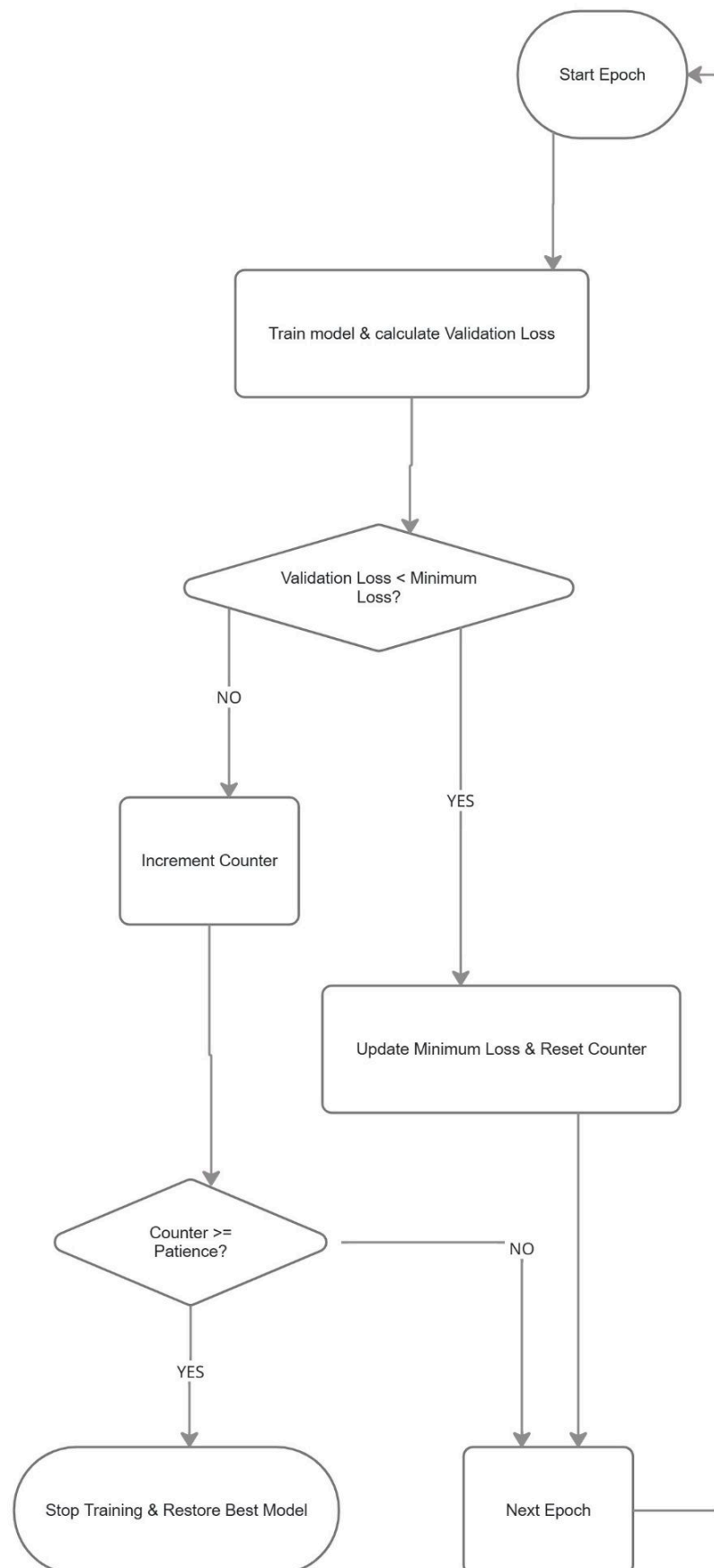
```
class EarlyStopper:
    def __init__(self, patience=1, min_delta=0):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.min_validation_loss = float('inf')

    def early_stop(self, validation_loss):
        if validation_loss < self.min_validation_loss:
            self.min_validation_loss = validation_loss
            self.counter = 0
        elif validation_loss > (self.min_validation_loss + self.min_delta):
            self.counter += 1
            if self.counter >= self.patience:
                return True
        return False
```

The early stopper acts as a monitor during the training loop. Its logic typically follows this sequence of tasks:

1. Set a **patience** value (the number of epochs to wait for improvement) and a **min_delta** (the minimum change to qualify as an improvement).
2. At the end of every training epoch, the validation loss is calculated and passed to the early stopper.
3. If the current validation loss is lower than the previously recorded minimum loss, reset the counter and save the current loss as the new minimum.
 - If the current validation loss is greater than the minimum loss (plus **min_delta**), increment the "patience" counter.
4. If the counter reaches the **patience** limit, the early stopper signals the training loop to break, preventing further overfitting.

HW3: Classification



Question 2

Explain how you will extend the neural network and select the different neural network configurations using validation technique.

- i. Provide the code to extend the number of layers to 1, 2, 3.

Answer

```
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def __init__(self, n_inputs, n_outputs=1, n_hidden=1, num_layers=1,
                  learning_rate=0.001, max_epoch=1000, model_folder="NeuralNet", patience=1):
        super(NeuralNetwork, self).__init__()

        # =====
        # HIDDEN LAYER CONFIGURATION BLOCK
        # =====

        # 1. When the number of layers is 1
        if num_layers == 1:
            model_list = []
            model_list.append(nn.Linear(n_inputs, n_outputs))
            model_list.append(nn.Sigmoid())
            self.model = nn.Sequential(*model_list)

        # 2. When the number of layers is 2
        elif num_layers == 2:
            model_list = []
            model_list.append(nn.Linear(n_inputs, n_hidden))
            model_list.append(nn.Linear(n_hidden, n_outputs))
            model_list.append(nn.Sigmoid())
            self.model = nn.Sequential(*model_list)

        # 3. When the number of layers is 3 or more
        else:
            model_list = []
            model_list.append(nn.Linear(n_inputs, n_hidden))
            for i in range(num_layers-2):
                model_list.append(nn.Linear(n_hidden, n_hidden))
            model_list.append(nn.Linear(n_hidden, n_outputs))
            model_list.append(nn.Sigmoid())
            self.model = nn.Sequential(*model_list)
```

```
# =====  
  
# ... [Other initializations: criterion, optimizer, EarlyStopper  
are omitted here] ...  
  
# ... [def forward(self, x): omitted] ...  
  
# ... [def train(self, ...): omitted] ...
```

1. When the number of layers is 1 (**num_layers == 1**):

It creates the simplest model configuration (similar to Logistic Regression). It uses a single **nn.Linear** function to map the input features directly to the output. After that, it applies the Sigmoid activation function (**nn.Sigmoid()**) to convert the final output into a probability between 0 and 1.

2. When the number of layers is 2 (**num_layers == 2**):

It extends the network by adding one hidden layer. First, an **nn.Linear** function connects the inputs to a hidden layer (**n_hidden** neurons). Then, a second **nn.Linear** function connects the hidden layer to the final output. Finally, it uses the Sigmoid activation function at the very end to format the prediction.

3. When the number of layers is 3 or more (**else** statement):

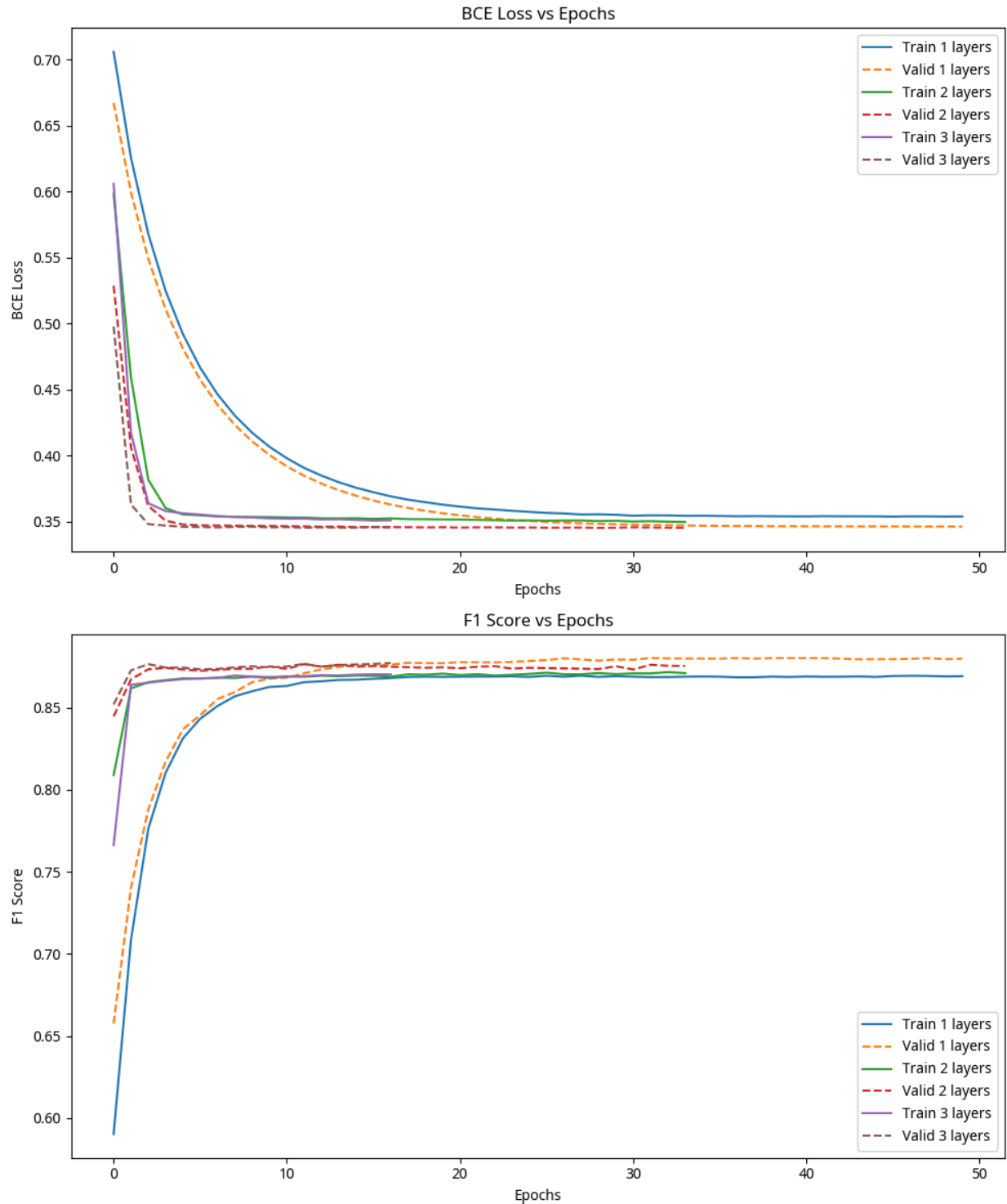
It creates a deep neural network. It starts with an initial **nn.Linear** function (input to hidden layer). Then, it uses a **for** loop (**range(num_layers-2)**) to automatically append the required number of intermediate hidden layers (connecting **n_hidden** to **n_hidden**). Finally, it adds the last **nn.Linear** layer to get the output, followed by the Sigmoid activation function.

HW3: Classification

- ii. Plot F1 / BCE loss curves versus the number of layers. Then, explain which you would use.

Answer

The following curves show the training and validation performance for 1, 2, and 3-layer configurations.



HW3: Classification

The Result is following:

num_layer	Final Valid BCE Loss	Final Valid F1 Score
1	0.3464	0.8798
2	0.3454	0.8753
3	0.3457	0.8771

Based on the experimental results:

1. The 2-layer and 3-layer models **converge significantly** faster than the 1-layer model. This is because the non-linear hidden layers allow the network to learn complex feature representations more efficiently.
2. All three configurations achieve very similar final F1 scores (around 0.87-0.88) and BCE losses (around 0.345).
3. I would select the **2-layer configuration** ,because It provides a significant improvement in convergence speed over the 1-layer model while maintaining a simpler architecture than the 3-layer model. In this specific dataset, adding a third layer does not yield a meaningful performance gain, suggesting that the extra complexity is unnecessary and might lead to overfitting on smaller datasets. The 2-layer model strikes the best balance between model capacity and simplicity

HW3: Classification

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

class NeuralNetwork(nn.Module):
    def __init__(self, n_inputs, n_outputs=1, n_hidden=16, num_layers=3):
        super(NeuralNetwork, self).__init__()
        layers = []
        # Layer 1: Input -> Hidden
        layers.append(nn.Linear(n_inputs, n_hidden))
        layers.append(nn.ReLU())

        # Layer 2: Hidden -> Hidden
        for _ in range(num_layers - 2):
            layers.append(nn.Linear(n_hidden, n_hidden))
            layers.append(nn.ReLU())

        # Layer 3: Hidden -> Output
        layers.append(nn.Linear(n_hidden, n_outputs))
        layers.append(nn.Sigmoid())

        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

def train_and_plot(train_set, valid_set, n_inputs):
    model = NeuralNetwork(n_inputs=n_inputs, num_layers=3, n_hidden=16)
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    epochs = 50
    batch_size = 256

    train_loader = DataLoader(train_set, batch_size=batch_size,
                              shuffle=True)
    valid_loader = DataLoader(valid_set, batch_size=batch_size,
                              shuffle=False)

    history = {'train_loss': [], 'valid_loss': [], 'train_f1': [],
              'valid_f1': []}
```

HW3: Classification

```
for epoch in range(epochs):
    model.train()
    train_losses, train_f1s = [], []
    for inputs, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        preds = (outputs > 0.5).float()
        tp = (preds * labels).sum().item()
        fp = (preds * (1 - labels)).sum().item()
        fn = ((1 - preds) * labels).sum().item()
        f1 = 2*tp / (2*tp + fp + fn + 1e-8)
        train_f1s.append(f1)

    model.eval()
    valid_losses, valid_f1s = [], []
    with torch.no_grad():
        for inputs, labels in valid_loader:
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            valid_losses.append(loss.item())
            preds = (outputs >= 0.5).float()
            tp = (preds * labels).sum().item()
            fp = (preds * (1 - labels)).sum().item()
            fn = ((1 - preds) * labels).sum().item()
            f1 = 2*tp / (2*tp + fp + fn + 1e-8)
            valid_f1s.append(f1)

    history['train_loss'].append(np.mean(train_losses))
    history['valid_loss'].append(np.mean(valid_losses))
    history['train_f1'].append(np.mean(train_f1s))
    history['valid_f1'].append(np.mean(valid_f1s))

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history['train_loss'], label='Train Loss')
plt.plot(history['valid_loss'], label='Valid Loss', linestyle='--')
plt.title('BCE Loss (3 Layers)')
```

HW3: Classification

```
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history['train_f1'], label='Train F1')
plt.plot(history['valid_f1'], label='Valid F1', linestyle='--')
plt.title('F1 Score (3 Layers)')
plt.legend()
plt.show()
```